

Oracle Database 11g: Administration

Support de cours - Module I

D50079EN10

Edition 1.0

June 2012

D53581



Plan du cours

- 1 **PRESENTATION GENERAL D'ORACLE**_____
- 2 **ARCHITECTURE D'UN SERVEUR ORACLE** _____
- 3 **REVISION SUR LE LANGAGE SQL** _____
- 4 **INSTALLATION ET CREATION D'UNE BASE DE DONNEES ORACLE**_____
- 5 **ADMINISTRATION D'UNE BASE DE DONNEES ORACLE**_____
- 6 **GESTION DES UTILISATEURS**_____
- 7 **GESTION DES OBJECTS** _____
- 8 **GESTION DU STOCKAGE D'UNE BASE DE DONNEES**_____
- 9 **UTILISATION DU RECOVERY MANAGER (RMAN)** _____
- 10 **SECURITE ET SURVEILLANCE DE LA BASE DE DONNEES**_____
- 11 **MINI PROJET**_____

1

Présentation General d'oracle

ORACLE

Copyright © 2007, Oracle. Tous droits réservés.

- **Présentation General d'oracle**

Oracle est un System de Gestion de Base de données relationnelle

(**SGBDR**) crée par la société **Oracle Corporation** en **1977** par Lawrence Ellison, Bob Minner et Ed OATS.

Relation Software Incorporation (RSI) est devenue Oracle Corporation en 1983.

Les différentes Version d'Oracle sont :

- **RDBMS-RSI1**
- **RDBMS-RSI2**
- **RDBMS-RSI3**
- **Oracle4, 5, 6, 7, 8,8i, 9i, 10g, 11g,12c**

Les différentes Editions sont :

- **Edition Standard**
- **Edition Personnel**
- **Edition Entreprise**

La base de données Oracle

Le système de gestion de base de données relationnelle (SGBDR) fourni par Oracle est un système de gestion de Base de données qui fournit une approche intégrée, Complète et ouverte de la gestion des informations.



La base de données Oracle

Une base de données est un ensemble de données traitées comme une seule et même unité. La fonction principale d'une base de données consiste à stocker et à extraire des informations associées.

Oracle Database gère de manière fiable une grande quantité de données dans un environnement multiutilisateur, permettant ainsi à de nombreux utilisateurs d'accéder simultanément aux mêmes données. Les opérations effectuées par les utilisateurs ne nuisent pas à ses performances. En outre, son mode de fonctionnement empêche tout accès non autorisé et fournit des solutions efficaces pour la récupération des données après incident.

Différents outils d'oracle

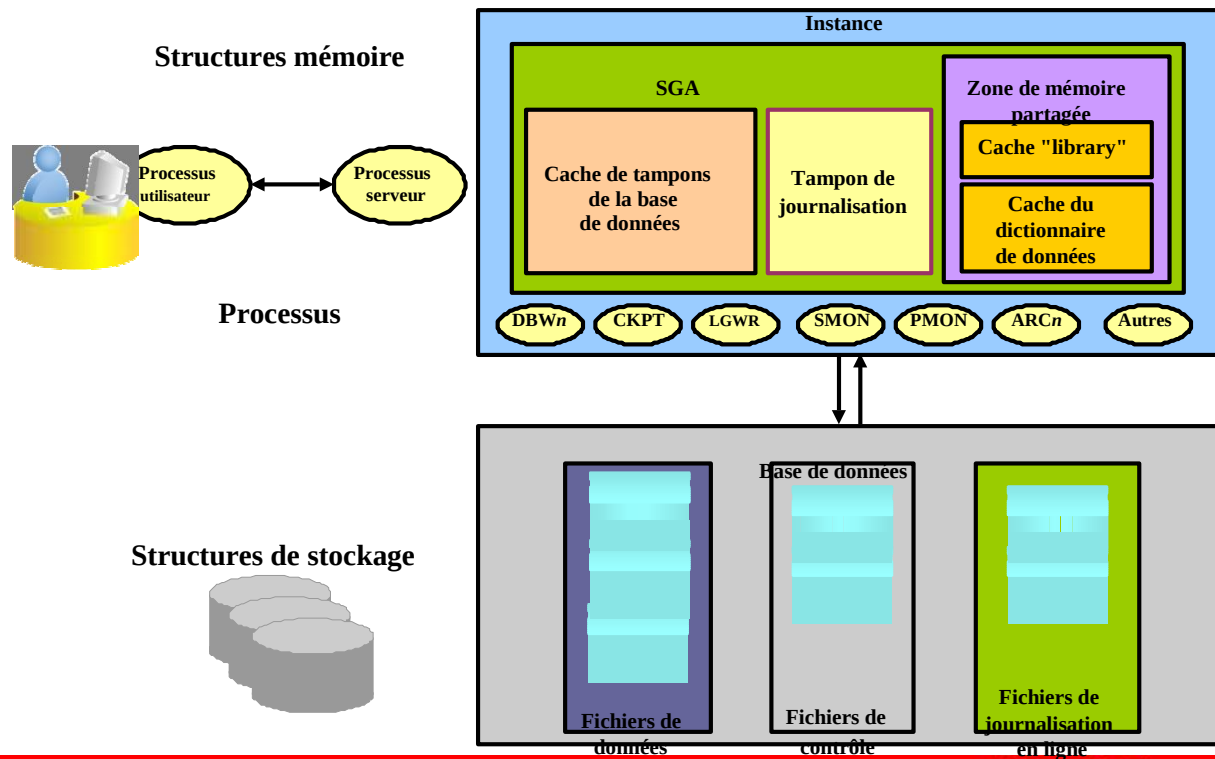
- Outils d'administration (Enterprise Manager)
- Outils de développement (Developer 2000, JDeveloper)
- Outils de Communication (SQLNET)
- Outils d'aide à la décision

2

Architecture du Serveur Oracle

ORACLE

Structures de serveur de base de données Oracle



Architecture de base de données Oracle

Un serveur Oracle est constitué d'une **instance Oracle** et d'une **base de données Oracle**.

1- Oracle Instance

L'instance Oracle comprend une région de la mémoire appelée La **SGA (System Global Area)**. La SGA contient des données et des informations de contrôle sur le serveur Oracle.

La mémoire est allouée à la SGA lors du démarrage d'une instance et dé-allouée lors de son arrêt. Cette zone mémoire est partagée et réside dans la mémoire vive qui n'est pas swappée ni paginée pour des raisons de performances.

La SGA se compose de plusieurs structures de groupe de mémoire, les trois principales étant :

Le pool partagé (Shared Pool) : Contient les requêtes SQL le plus récemment exécuté et L'information du dictionnaire de données la plus récemment utilisée.

Le buffer de cache de la base de données (Database Buffer Cache) : Cette structure stocke les données utilisatrices les plus récentes tirées des tables.

Le buffer de redo Log (Redo Log Buffer) : Ce buffer est utilisé pour enregistrer les modifications effectuées sur la base de données.

Process d'arrière plan

Une instance Oracle comprend, en plus de la SGA, un ensemble de process d'arrière plan qui sont au niveau du système d'exploitation.

Oracle 11g comprend cinq process d'arrière plan obligatoire pour une instance :

Database Writer (DBWn) : Il écrit les données chargées du buffer de cache de base de données vers les fichiers de données.

Log Writer (LGWR) : Il enregistre les changements qui sont instanciés dans le buffer de redo log vers les fichiers de redo log. (Ces changements sont référencés comme les données du redo log)

System Monitor (SMON) : Il exécute une récupération de l'instance lors de son démarrage, il nettoie les segments temporaires qui ne sont plus utilisés et annule les transaction qui n'avaient pas été validées lors d'un plantage du système. Il permet de vérifier la cohérence des données.

Process Monitor (PMON) : Il récupère les process lorsque qu'un process utilisateur échoue. Il est responsable du nettoyage des process et des ressources inutilisés et de déverrouiller les ressources système bloquées par un process utilisateur.

Check point (CKPT) : il met à jour les statuts d'information sur la base de données tels que les entêtes des fichiers de base de données. Ceci est fait lors d'un événement de check point tel qu'une rotation des logs. Les changements dans le buffer de cache de la base de données sont alors écrits de manière permanente dans les fichiers de données.

En plus de ces process obligatoires, on trouve un ensemble de process optionnels :

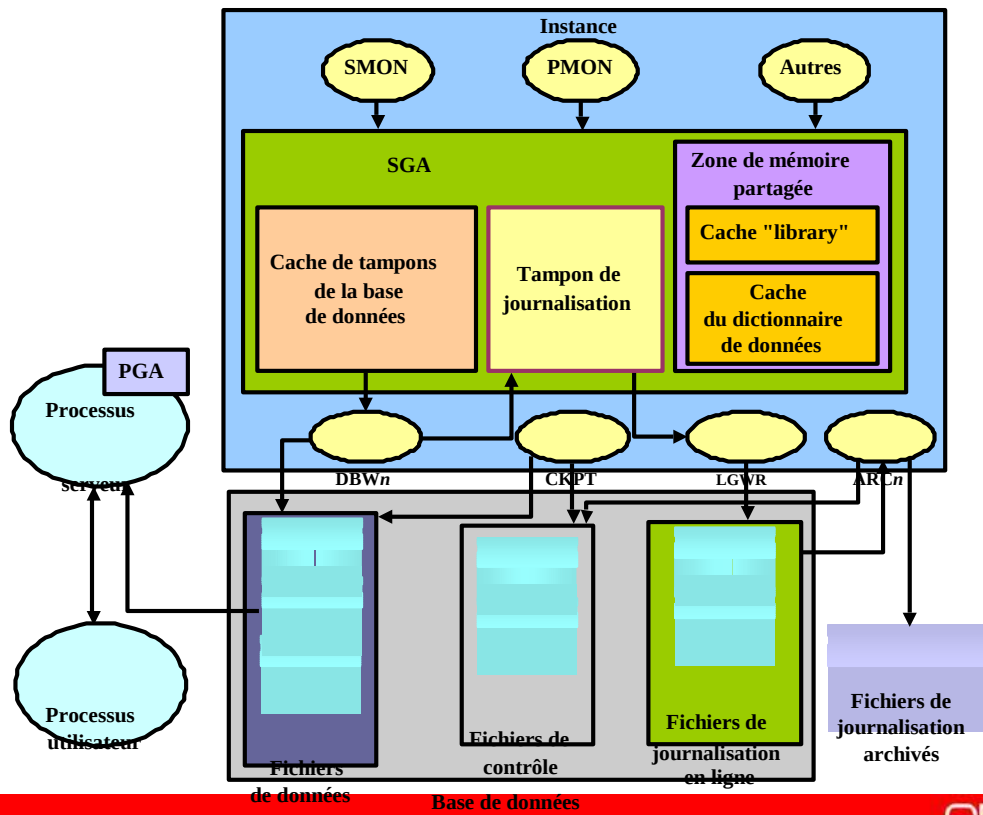
Recoverer (RECO)

Archiver (ARCn)

Lock (LCKn)

Dispatcher (Dnnn)

Architecture de base de données Oracle : Présentation



2- Fichiers de la Base de données

Une base de données Oracle est une collection de données qui stocke et fournit un ensemble d'informations communes.

Ces données sont stockées dans des fichiers physiques, le DBA est à la charge de ces fichiers de base de données.

Il existe trois types de fichiers dans une base de données Oracle :

Les fichiers de données (Data File) :

Il peut y avoir un ou plusieurs fichiers de données. Ces fichiers ont les rôles suivants :

Stocke le dictionnaire de données.

Stocke les objets utilisateurs.

Stocke les « image avant » des blocs de données qui sont modifiés par les transactions courantes.

Les fichiers de redo log (Redo Log File) :

Une base de données Oracle comporte au moins deux groupes de redo log, chaque groupe ayant au moins un fichier de redo log. Ces fichiers servent à enregistrer tous les changements effectués sur les données pour assurer leur restauration en cas de dysfonctionnement du système.

Pour renforcer leur tolérance à une panne de disque dur, Oracle supporte les fichiers de redo log multiplexés. On peut ainsi maintenir une ou plusieurs copie de fichiers de redo log sur différents disques (On parle de fichiers mirrorés).

Les fichiers de contrôle (Control File)

Ces fichiers contiennent les informations requises pour maintenir et vérifier l'intégrité de la base de données. Ils renferment la structure des données (Nom de la base, date de création, chemins vers les fichiers de données et de redo log). Une base de données Oracle nécessite au moins un fichier de contrôle, il est toutefois possible d'également les mirrorer sur plusieurs disques dur.

3- Fichiers relatant à la Base de données

Ce sont les fichiers qui ne font pas partie de la base de données mais qui sont utilisés par le serveur Oracle pour des opérations telles que le démarrage d'une instance ou l'authentification des utilisateurs.

Il existe trois types de fichiers :

Le fichier de paramètre (Parameter File) :

Il va définir les caractéristiques de l'instance. Lors du démarrage d'une instance, le serveur Oracle va lire les paramètres d'initialisation du fichier de paramètres.

On retrouve des informations telles que le nom de la base de données, la taille de la mémoire à allouer, le nom des fichiers de contrôle ainsi que les autres paramètres système.

Le fichier de mots de passe (Password File) :

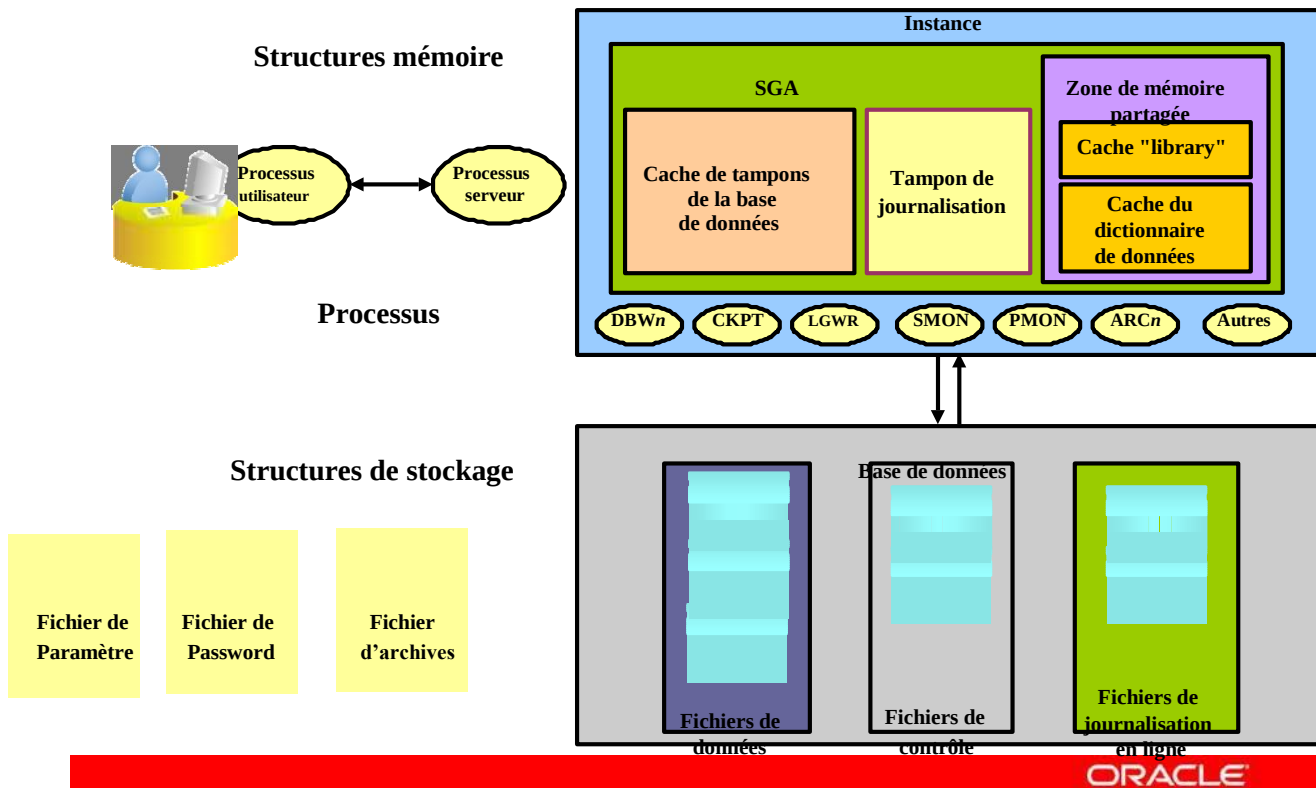
Il est utilisé pour authentifier les utilisateurs privilégiés.

Il permet à sysdba/sysoper/system de se connecter à distance à la base de données et d'effectuer des tâches d'administration

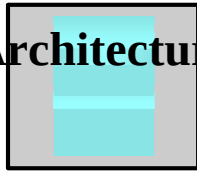
Les fichiers de redo log archivés (Archived Redo Log Files) :

Ils contiennent des copies off line des fichiers de redo log. Ils sont utilisés lors d'une récupération suite à une défaillance d'un média. Les fichiers de redo log sont sauvés lorsqu'ils sont pleins, cette fonctionnalité n'est utilisée que lorsque la base de données est en mode "ARCHIVELOG".

Structures de serveur de base de données Oracle



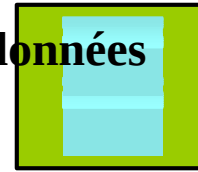
Architecture de stockage de la base de données



Fichiers de
contrôle



Fichiers de
données



Fichiers de
journalisation en ligne

Fichier de
paramètres



Fichiers de
sauvegarde



Fichiers de
journalisation
archivés

Fichier de
mots de passe

Fichiers d'alertes
et fichiers trace

ORACLE

Architecture de stockage de la base de données

Les fichiers constituant une base de données Oracle sont organisés de la façon suivante :

- **Fichiers de contrôle** : Ils contiennent des données sur la base elle-même (informations sur la structure physique de la base de données). Ces fichiers sont d'une importance capitale pour la base. Sans eux, vous ne pouvez pas ouvrir de fichiers de données pour accéder aux données de la base.
- **Fichiers de données** : Ils contiennent les données utilisateur ou données des applications de la base de données, ainsi que des métadonnées et le dictionnaire de données.
- **Fichiers de journalisation en ligne (online redo logs)** : Ils permettent la récupération d'une instance de base de données. Si le serveur de base de données connaît une défaillance et qu'aucun fichier de données n'est perdu, l'instance peut récupérer la base de données à partir des informations contenues dans ces fichiers.

Les autres fichiers indiqués ci-dessous sont essentiels au bon fonctionnement de la base de données :

- **Fichier de paramètres** : Il permet de définir la façon dont l'instance est configurée au démarrage.

- **Fichier de mots de passe** : Il permet à sysdba/sysoper de se connecter à distance à la base de données et d'effectuer des tâches d'administration.

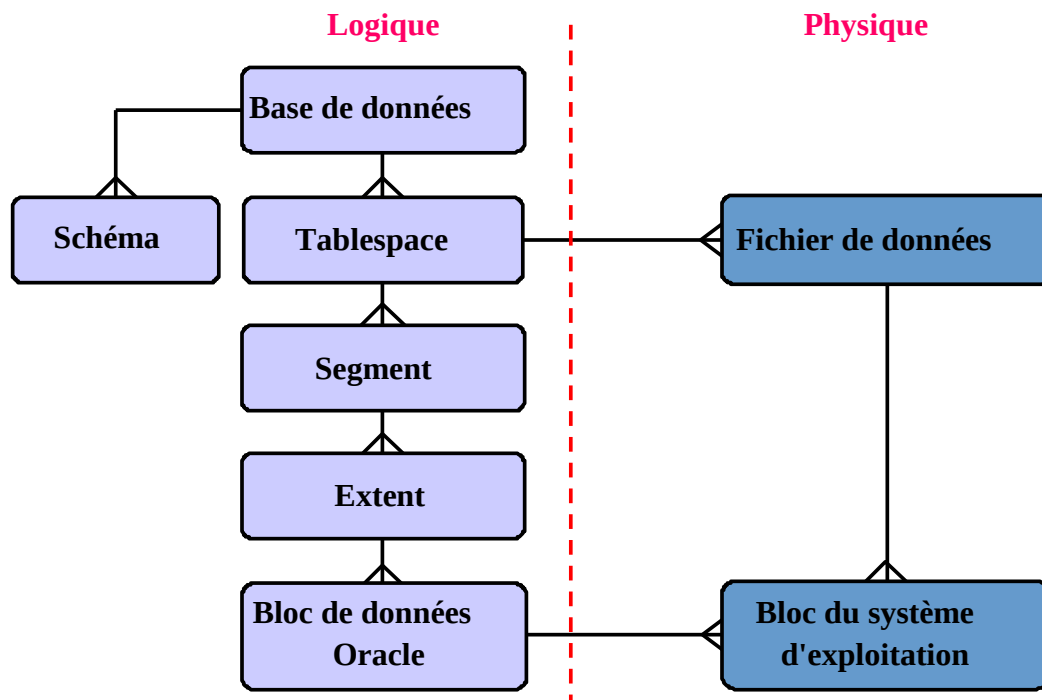
- **Fichiers de sauvegarde** : Ils sont utilisés pour la récupération de la base de données. En règle générale, vous restaurez un fichier de sauvegarde lorsqu'une défaillance Physique ou une erreur utilisateur a endommagé ou supprimé le fichier d'origine

. **Fichiers de journalisation archivés (archived redo logs)** : Ils contiennent l'historique complet des modifications de données (informations de journalisation) générées par l'instance. A l'aide de ces fichiers et d'une sauvegarde de la base de données, vous pouvez récupérer un fichier de données perdu. Autrement dit, les fichiers de journalisation archivés permettent la récupération des fichiers de données restaurés.

- **Fichiers trace** : Chaque processus serveur et processus en arrière-plan peut écrire dans un fichier trace associé. Lorsqu'un processus détecte une erreur interne, il réalise dans son fichier trace un dump des informations relatives à cette erreur. Certaines informations écrites dans un fichier trace sont destinées à l'administrateur de base de données, et d'autres au Support technique Oracle.

- **Fichier d'alertes** : Il contient des entrées de trace spéciales. Le fichier d'alertes d'une base de données est un journal chronologique des messages et des erreurs. Il existe un fichier d'alertes pour chaque instance. Oracle recommande de le consulter régulièrement.

Structures logiques et structures physiques d'une base de données



ORACLE

Structures logiques et structures physiques d'une base de données

Une base de données comporte des structures logiques et des structures physiques.

- **Structures physiques**

Ce sont des structures qui existent physiquement sur le disque :

- Les Fichiers de données (Data files)
- Les Fichiers de Contrôles (Contrôle files)
- Les Fichiers de Journalisations (Redo Log files)

- **Structures Logiques**

Ce sont des structures qui ne sont pas matérialisés physiquement mais qui constituent des blocs de regroupement de données :

- Les Datablocks
- Les Extents
- Les Segments
- Les Tablespaces
- Les objets du Schema

1- Blocs de données

Au niveau de détail le plus fin, les données d'une base Oracle sont stockées dans des blocs de données. Un bloc de données correspond à un nombre précis d'octets d'espace physique sur le disque. Chaque base utilise et alloue de l'espace libre de base de données dans les blocs de données Oracle.

2- Extents

Le terme extent (ensemble de blocs contigus) désigne le niveau suivant d'espace de base de données logique. Un extent consiste en un nombre défini de blocs de données contigus (obtenus par une allocation unique) qui permettent de stocker un type spécifique d'informations.

3- Segments

Le niveau logique de stockage situé au-dessus d'un extent s'appelle un segment. Un segment est un ensemble d'extents alloués à une structure logique donnée. On rencontre notamment les types de segment suivants :

- **Segments de données :**

Toutes les tables de données sont stockées dans ce segment.

- **Segments d'index :**

Chaque index comporte un segment d'index dans lequel sont stockées toutes ses données relatives à l'index.

- **Segments d'annulation (undo) :** Un tablespace UNDO est créé pour chaque instance de base de données ; il contient de nombreux segments d'annulation permettant de stocker temporairement des informations d'annulation. Les informations contenues dans un segment d'annulation permettent de générer des informations de base de données cohérentes en lecture et, lors de la récupération de la base, d'annuler les transactions non validées pour les utilisateurs.

- **Segments temporaires :** Ces segments sont créés par la base de données Oracle lorsque l'exécution d'une instruction SQL requiert une zone de travail temporaire. Une fois l'instruction exécutée, les extents du segment temporaire sont renvoyés à l'instance en vue d'une utilisation ultérieure. Vous pouvez indiquer un tablespace temporaire par défaut pour chaque utilisateur ou à l'échelle de la base de données.

4- Tablespaces

Une base de données est divisée en unités de stockage logiques appelées tablespaces, qui regroupent des structures logiques associées. Par exemple, les tablespaces regroupent habituellement tous les objets d'une application afin de simplifier certaines opérations d'administration. Vous pouvez disposer d'un premier tablespace pour les données d'application et d'un second pour les index d'application.

5- Schémas

Un schéma est un ensemble d'objets de base de données appartenant à un utilisateur de la base. Les objets de schéma sont les structures logiques qui font directement référence aux données de la base. Ils incluent des structures telles que des tables, des vues, des séquences, des procédures stockées, des synonymes, des index, des clusters et des liens de base de données.

Table : c'est une entité dont les informations sont stockées dans des colonnes et des lignes. Ces colonnes ont des noms, type de données et des tailles

Vue : c'est le résultat stocké d'une requête et elle n'a pas d'existence propre.

Séquence : c'est un objet qui permet de générer une série numérique.

Procédure stockée : c'est un ensemble d'ordre SQL et PL/SQL (programme) qui réalise une tâche.

Index : c'est une clé pour avoir accès rapidement à des données d'une table

Database link : c'est un lien qui permet de connaître le chemin d'une Base de Données.

3

REVISION SUR LE LANGAGE SQL

ORACLE

1-Définition des acronymes

SQL :Structured Query Langage

LDD (Langage de Définition de Données) :

création, modification et suppression des définitions des tables

LMD (Langage de Manipulation de Données) :

ajout, suppression, modification et interrogation des données

LCD (Langage de Contrôle de Données) : gestion

des protections d'accès

- *Fin d'instruction : ;*
- *Commentaires : /* ... */ ou –commentaire*

Base de données exemple

CLIENT (NumCli, Nom, Prénom, DateNaiss, Rue,CP, Ville)

PRODUIT (NumProd, Dési, PrixUni, NumFour)

FOURNISSEUR (NumFour, RaisonSoc)

COMMANDE (NumCli, NumProd, Date, Quantité)

Clés primaires

Clés étrangères

2. Définition des données

Définitions des tables

```
CREATE TABLE (Attribut1 TYPE, Attribut2 TYPE, ...,  
               contrainte_integrité1,  
               contrainte_integrité2, ...);
```

Type des données :

- NUMBER(n) : Entier à n chiffres
- NUMBER(n, m) : Réel à n chiffres au total
(virgule comprise), m après la virgule
- VARCHAR(n) : Chaîne de n caractères (entre ' ')
- DATE : Date au format 'JJ-MM-AAAA'

Définitions des contraintes d'intégrité

- *Clé primaire :*

```
CONSTRAINT nom_contrainte PRIMARY KEY  
(attribut_clé [, attribut_clé2, ...])
```

- *Clé étrangère :*

```
CONSTRAINT nom_contrainte FOREIGN KEY  
(attribut_clé_ét) REFERENCES table(attribut)
```

- *Contrainte de domaine :*

```
CONSTRAINT nom_contrainte CHECK (condition)
```


Définition des données

```
ex. CREATE TABLE Client (  
    — NumCli NUMBER(3),  
    Nom CHAR(30),  
    DateNaiss DATE,  
    Salaire NUMBER(8,2),  
    NumEmp NUMBER(3),  
  
    CONSTRAINT cle_pri PRIMARY KEY (NumCli),  
    CONSTRAINT cle_etr FOREIGN KEY (NumEmp)  
        REFERENCES EMPLOYEUR(NumEmp),  
    CONSTRAINT date_ok CHECK (DateNaiss < SYSDATE));
```

- **Création d'index** (accélération des accès)

```
CREATE [UNIQUE] INDEX nom_index ON  
nom_table (attribut [ASC|DESC], ...);
```

UNIQUE ® pas de double

ASC/DESC ® ordre croissant ou décroissant

```
ex. CREATE UNIQUE INDEX Icli ON Client (NumCli);
```

- **Destructions :**
DROP TABLE nom_table;
DROP INDEX nom_index

- **Ajout d'attributs**

ALTER TABLE nom_table ADD (attribut TYPE, ...);

ex. ALTER TABLE Client ADD (tel NUMBER(8));

- **Modifications d'attributs**

ALTER TABLE nom_table MODIFY (attribut TYPE, ...);

ex. ALTER TABLE Client MODIFY (tel NUMBER(10));

- **Suppression de contraintes**

ALTER TABLE nom_table DROP CONSTRAINT

nom_contrainte;



3. Mise à jour des données

- **Ajout d'un tuple**

INSERT INTO nom_table

VALUES (val_att1, val_att2, ...);

ex. INSERT INTO Produit

VALUES (400, 'Nouveau produit', 78.90);

- **Mise à jour d'un attribut**

UPDATE nom_table SET attribut=valeur

[WHERE condition];

ex. UPDATE Client SET Nom='Dudule'

WHERE NumCli = 3;

3. Mise à jour des données

- **Suppression de tuples**

DELETE FROM nom_table [WHERE condition];

ex. DELETE FROM Produit;

DELETE FROM Client
WHERE Ville = 'Lyon';

4. Interrogation des données

SELECT [ALL|DISTINCT] attribut(s) FROM table(s)
[WHERE condition]
[GROUP BY attribut(s) [HAVING condition]]
[ORDER BY attribut(s) [ASC|DESC]];

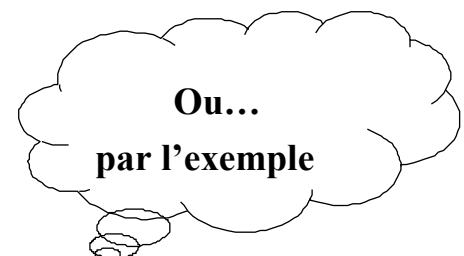
- **Tous les tuples d'une table**

ex. SELECT * FROM Client;

- **Tri du résultat**

ex. *Par ordre alphabétique inverse de nom*

SELECT * FROM Client
ORDER BY Nom DESC;



4. Interrogation des données

- **Calculs** *ex. Calcul de prix TTC*

```
SELECT PrixUni+PrixUni*0.18 FROM Produit;
```

- **Projection**

ex. Noms et Prénoms des clients, uniquement

```
SELECT Nom, Prenom FROM Client;
```

- **Restriction**

ex. Clients qui habitent à Lyon

```
SELECT * FROM Client  
WHERE Ville = 'Lyon';
```

II.4. Interrogation des données

ex. Commandes en quantité au moins égale à 3

```
SELECT * FROM Commande  
WHERE Quantite >= 3;
```

ex. Produits dont le prix est compris entre 50 et 100 F

```
SELECT * FROM Produit  
WHERE PrixUni BETWEEN 50 AND 100;
```

ex. Commandes en quantité indéterminée

```
SELECT * FROM Commande  
WHERE Quantite IS NULL;
```


4. Interrogation des données

ex. *Clients habitant une ville dont le nom se termine par sur-Saône*

```
SELECT * FROM Client  
WHERE Ville LIKE '%sur-Saône';
```

'sur-Saône%' ® commence par *sur-Saône*

'%sur%' ® contient le mot *sur*

4. Interrogation des données

ex. *Prénoms des clients dont le nom est Dupont, Durand ou Martin*

```
SELECT Prenom FROM Client  
WHERE Nom IN ('Dupont', 'Durand', 'Martin');
```

~~NB~~ : Possibilité d'utiliser la négation pour tous ces prédicats : **NOT BETWEEN, NOT NULL, NOT LIKE, NOT IN.**

4. Interrogation des données

- **Fonctions d'agrégat**

Elles opèrent sur un ensemble de valeurs.

- AVG(), VARIANCE(), STDDEV() : moyenne, variance et écart-type des valeurs
- SUM() : somme des valeurs
- MIN(), MAX() : valeur minimum, valeur maximum
- COUNT() : nombre de valeurs

ex. *Moyenne des prix des produits*

```
SELECT AVG(PrixUni) FROM Produit;
```

4. Interrogation des données

Opérateur DISTINCT

ex. *Nombre total de commandes*

```
SELECT COUNT(*) FROM Commande;
```

```
SELECT COUNT(NumCli) FROM Commande;
```

ex. *Nombre de clients ayant passé commande*

```
SELECT COUNT( DISTINCT NumCli)
```

```
FROM Commande;
```


4. Interrogation des données

Table COMMANDE (simplifiée)

<i>NumCli</i>	<i>Date</i>	<i>Quantite</i>
1	22/09/99	1
3	22/09/99	5
3	22/09/99	2

COUNT(NumCli) ® Résultat = 3

COUNT(DISTINCT NumCli) ® Résultat = 2

4. Interrogation des données

- Jointure

ex. Liste des commandes avec le nom des clients

```
SELECT Nom, Date, Quantite
FROM Client, Commande
WHERE Client.NumCli =Commande.NumCli;
```

4. Interrogation des données

ex. *Idem avec le numéro de client en plus*

```
SELECT C1.NumCli, Nom, Date, Quantite
FROM Client C1, Commande C2
WHERE C1.NumCli = C2.NumCli
ORDER BY Nom;
```

NB : Utilisation d'*alias* (C1 et C2) pour alléger
l'écriture + tri par nom.

II.4. Interrogation des données

~~Jointure exprimée avec le prédicat IN~~

~~ex.~~ *Nom des clients qui ont commandé le 23/09*

```
SELECT Nom FROM Client
WHERE NumCli IN (
    SELECT NumCli FROM Commande
    WHERE Date = '23-09-1999'
);
```


NB : Il est possible d'imbriquer des requêtes.

II.4. Interrogation des données

- **Prédicats EXISTS / NOT EXISTS**

ex. *Clients qui ont passé au moins une commande*
[n'ont passé aucune commande]

```
SELECT * FROM Client C1
WHERE [NOT] EXISTS (
    SELECT * FROM Commande C2
    WHERE C1.NumCli = C2.NumCli);
```

4. Interrogation des données

- **Prédicats ALL / ANY**

ex. *Numéros des clients qui ont commandé au moins un produit en quantité supérieure à chacune [à au moins une] des quantités commandées par le client n° 1.*

- ```
SELECT DISTINCT NumCli FROM Commande
WHERE Quantite > ALL [ANY] (
 SELECT Quantite FROM Commande
 WHERE NumCli = 1);
```

## 4. Interrogation des données

- **Groupement**

ex. *Quantité totale commandée par chaque client*

```
SELECT NumCli, SUM(Quantite)
FROM Commande
GROUP BY NumCli;
```

ex. *Nombre de produits différents commandés...*

```
SELECT NumCli, COUNT(DISTINCT NumProd)
FROM Commande
GROUP BY NumCli;
```

## II.4. Interrogation des données

~~ex.~~ *Quantité moyenne commandée pour les  
produits faisant l'objet de plus de 3  
commandes*

```
SELECT NumProd, AVG(Quantite)
FROM Commande
GROUP BY NumProd
HAVING COUNT(*)>3;
```

~~Attention :~~ La clause HAVING ne s'utilise qu'avec  
GROUP BY.

## II.4. Interrogation des données

- **Opérations ensemblistes**

INTERSECT, MINUS, UNION

*ex. Numéro des produits qui soit ont un prix  
inférieur à 100 F, soit ont été commandés  
par le client n° 2*

```
SELECT NumProd FROM Produit WHERE PrixUni<100
UNION
SELECT NumProd FROM Commande WHERE NumCLi=2;
```

## 4. Interrogation des données

- **Fonctions SQL\*Plus**

- |                                         |                                                   |
|-----------------------------------------|---------------------------------------------------|
| – ABS(n) : Valeur absolue de n          | – LOWER(ch) : c en minuscules                     |
| – CEIL(n) : Plus petit entier $\geq$ n  | – UPPER(ch) : c en majuscules                     |
| – FLOOR(n) : Plus grand entier $\leq$ n | – LTRIM(ch, n) : Troncature à gauche              |
| – MOD(m, n) : Reste de m/n              | – RTRIM(ch, n) : Troncature à droite              |
| – POWER(m, n) : $m^n$                   | – REPLACE(ch, car) :<br>Remplacement de caractère |
| – SIGN(n) : Signe de n                  | – SUBSTR(ch, pos, lg) : Extraction<br>de chaîne   |
| – SQRT(n) : Racine carrée de n          | – SOUNDEx(ch) : Cp. Phonétique                    |
| – ROUND(n, m) : Arrondi à 10-m          | – LPAD(ch, lg, car) : Compléter à<br>gauche       |
| – TRUNC(n, m) : Troncature à 10-m       | – RPAD(ch, lg, car) : Compléter à<br>droite       |
| – CHR(n) : Caractère ASCII n° n         |                                                   |
| – INITCAP(ch) : 1ère lettre en maj.     |                                                   |

#### 4. Interrogation des données

- |                                                                                                                                                                                                                                                                                                                                                                                                                                          |                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>– ASCII(ch) : Valeur ASCII de ch</li><li>– INSTR(ch, ssch) : Recherche de ssch dans ch</li><li>– LENGTH(ch) : Longueur de ch</li><li>– ADD_MONTHS(dte, n) : Ajout de n mois à dte</li><li>– LAST_DAY(dte) : Dernier jour du mois</li><li>– MONTHS_BETWEEN(dt1, dt2) : Nombre de mois entre dt1 et dt2</li><li>– NEXT_DAY(dte) : Date du lendemain</li><li>– SYSDATE : Date/heure système</li></ul> | <ul style="list-style-type: none"><li>– TO_NUMBER(ch) : Conversion de ch en nombre</li><li>– TO_CHAR(x) : Conversion de x en chaîne</li><li>– TO_DATE(ch) : Conversion de ch en date</li><li>– NVL(x, val) : Remplace par val si x a la valeur NULL</li><li>– GREATEST(n1, n2...) : + grand</li><li>– LEAST (n1, n2...) : + petit</li><li>– UID : Identifiant numérique de l'utilisateur</li><li>– USER : Nom de l'utilisateur</li></ul> |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

#### EXERCICES

## Exercice 1

Soit le schéma relationnel de la base de données « pilotes-avions-vols ».

PILOTE (PLNUM, PLNOM, PLPRENOM, VILLE, SALAIRE)

AVION (AVNUM, AVNOM, CAPACITE, LOCALISATION)

VOL (VOLNUM, PLNUM, AVNUM, VILLEDEP, VILLEARR, HEUREDEP, HEUREARR)

Exprimer les requêtes suivantes en SQL\*Plus.

- 1) Liste de tous les vols.
- 2) Nom, prénom et ville de tous les pilotes, par ordre alphabétique.
- 3) Nom, prénom et salaire des pilotes dont le salaire est supérieur à 20 000 F.
- 4) Numéro et nom des avions localisés à Paris.
- 5) Caractéristiques (VOLNUM, VILLEDEP, VILLEARR, HEUREDEP, HEUREARR, AVNOM, PLNOM) du vol numéro 714.
- 6) Nom, prénom et numéro de vol des pilotes affectés à un vol.
- 7) Numéro et nom des avions affectés à des vols.
- 8) Nombre total de vols.
- 9) Somme des capacités par type (nom) d'avion.
- 10) Moyenne des durées des voyages.

## Exercice 2

Sur la base de données exemple du cours (CLIENT-COMMANDE-PRODUIT-FOURNISSEUR), formuler avec le langage SQL\*Plus les requêtes suivantes.

- 1) Désignation et prix unitaire de tous les produits.
- 2) Désignation des produits de prix inférieur à 100 F.
- 3) Nom des clients qui ont commandé le produit n° 1.
- 4) Nom des clients qui ont commandé un produit de prix supérieur à 500 F.
- 5) Nom des clients qui n'ont pas commandé le produit n° 1.
- 6) Numéro et nom des clients qui habite Marseille ou Monaco.
- 7) Nom des Client qui commence par « A » et qui se termine par « E ».
- 8) Le Nombre total de produit commandé la ville



### Exercice 3

Soit le schéma relationnel de la base FABRICATION.

CLIENT(NOC, NOM, ADRESSE)

SERVICE(NOS, INTITULE, LOCALISATION)

PIECE(NOP, DESIGNATION, COULEUR, POIDS)

COMMANDE (NOS, NOC, NOP QUANTITE)

Formuler en SQL\*Plus les commandes de création de la structure de cette base, puis exprimer les requêtes suivantes.

- 1) Donner pour chaque service le poids de la pièce commandée de couleur bleue.
- 2) Donner le poids moyen des pièces commandées pour chacun des services "Promotion".
- 3) Donner le nombre total des pièces qui ont été commandées par le service localisé à Lyon.
- 4) Donner le maximum des quantités des pièces commandées par les différents services.

